

Beauty SaaS Platform

Module Workflow & Test Guide — Phases 1 through 8

Scope: Platform & Tenancy, Organization/Branches/Employees, Customer CRM, Service Catalogue & Vertical Consultations, Appointment Engine, POS/Invoicing, Inventory & Purchasing, Packages/Membership/Loyalty/Wallet/Gift Cards

Target market: India (GST-compliant invoicing, INR, IST)

How to use this document: Each module below lists the workflow(s) to walk through, which role can perform them, and the expected result at each step. Tick the checkbox column as you verify each step.

Contents

0. Getting Started — running the app & test accounts

1. Phase 1 — Platform & Tenancy

2. Phase 2 — Organization, Branches, Resources, Employees

3. Phase 3 — Customer CRM (DPDP consent & erasure)

4. Phase 4 — Service Catalogue & Vertical Consultations

5. Phase 5 — Appointment Engine

6. Phase 6 — POS, Invoicing, Payments, Refunds

7. Phase 7 — Products, Inventory & Purchasing

8. Phase 8 — Packages, Membership, Loyalty, Wallet, Gift Cards

9. Role & Permission Matrix (Owner / Manager / Staff)

10. Cross-Cutting Checks (tenant isolation, tax, financial integrity)

0. Getting Started

Running the application

Start the dev server from the project root, then open the URL in your browser:
`php artisan serve --port=8010` → `http://localhost:8010`

Test accounts

Super Admin (Platform Console)	
URL	<code>/platform/login</code>
Email	<code>admin@example.com</code>
Password	<code>password</code>

There is no pre-seeded tenant/business account — create your own via self-service registration at `/register` (this creates a tenant + an Owner account together). Use the Employees module afterwards to create Manager/Staff accounts for role-based testing.

General notes for testers

✓	Note
☐	This app targets India only — every tenant is fixed to IST timezone and INR currency automatically at signup (not asked on the form).
☐	All money values should always be server-calculated — never trust a total shown before you submit a form; verify the saved value matches expectation.
☐	Every list/detail page you can reach is also worth trying as a <i>different tenant's</i> user to confirm you get a 404/redirect, not the other tenant's data (see Section 10).
☐	Where a workflow says "Owner only", try it as Manager/Staff too and confirm you correctly get a 403 Forbidden.

1. Phase 1 — Platform & Tenancy

Super Admin

1.1 Tenant self-service signup

✓	#	Action	Expected result
☐	1	Go to <code>/register</code> . Fill business name, pick at least one module (Salon/Beauty Parlour/Spa), your name/email/password.	Form has no timezone/currency fields — India/INR is implicit.
☐	2	Submit.	New tenant + Owner account created, auto-logged in, redirected to <code>/dashboard</code> .
☐	3	Log out, log back in at <code>/login</code> with the same credentials.	Successful login, lands on dashboard.
☐	4	Try registering again with the <i>same email</i> .	Validation error — email already in use.

1.2 Super Admin — Tenants

✓	#	Action	Expected result
☐	1	Log in at <code>/platform/login</code> .	Platform dashboard shows Trial/Active/Suspended/Cancelled tenant counts + recent tenants.
☐	2	Go to Tenants → open the tenant you just created.	Shows business name, status, modules, subscription plan, users.
☐	3	Suspend the tenant (change status).	Status flips to Suspended.
☐	4	As that tenant's Owner, try to log in or refresh any tenant page.	Logged out automatically with "account is currently suspended" error.
☐	5	Reactivate the tenant from Platform.	Owner can log in again normally.
☐	6	Toggle a module on/off for the tenant (e.g. enable Spa).	Spa-specific nav/features appear for that tenant after refresh.
☐	7	From Platform, create a brand-new tenant directly via + New tenant .	Tenant + Owner created without the Owner ever visiting <code>/register</code> .

1.3 Super Admin — Subscription Plans

✓	#	Action	Expected result
☐	1	Go to Subscription Plans .	Lists Free Trial / Growth / Pro with price, billing interval, feature count, subscribed tenant count.
☐	2	Edit a plan — change price, toggle a feature checkbox, save.	Changes persist; feature list on the edit page reflects the new selection on reload.
☐	3	Un-check "Active" on a plan and save.	Plan shows Inactive on the index; existing tenant subscriptions to it are unaffected.

1.4 RBAC foundation

✓	#	Action	Expected result
☐	1	As Owner, go to Employees, create a new employee, assign role "Manager".	Manager can log in but sees a narrower sidebar (no Void/Refund, no template deletes).
☐	2	Create another employee with role "Staff".	Staff sees only read-mostly access (view lists, create appointments/customers) — most write/delete actions return 403 if attempted directly by URL.

2. Phase 2 — Organization, Branches, Resources, Employees

2.1 Organization Settings

Owner

✓	#	Action	Expected result
☐	1	Go to Settings . Update business profile (legal name, contact, cancellation policy).	Saved and reflected on reload.
☐	2	Set weekly business hours (open/close per day, mark one day closed).	Hours saved; the closed day later blocks appointment booking (see 5.1).
☐	3	Set Loyalty program rate (points per ₹100, redemption value) — see Phase 8.	Saved; loyalty becomes active tenant-wide.

2.2 Branches

✓	#	Action	Expected result
☐	1	Create a branch (name, code). Assign modules to it — try assigning a module the <i>tenant</i> doesn't have enabled.	Rejected — a branch's modules must be a subset of the tenant's enabled modules.
☐	2	Set branch-specific hours and add a holiday date.	Saved; that date later blocks booking at this branch only.
☐	3	Create a second branch.	Branch switcher/selector appears on Dashboard, Appointments, Inventory, etc.

2.3 Resources (chairs / rooms / stations)

✓	#	Action	Expected result
☐	1	Under a branch, add a resource (e.g. "Chair 1").	Resource created, available for appointment booking against that branch.
☐	2	Deactivate the resource.	No longer selectable for new bookings; historical appointments referencing it remain intact.

2.4 Employees

✓	#	Action	Expected result
☐	1	Create an employee, assign to one specific branch (not "all branches").	That employee cannot be booked/scheduled at any other branch.
☐	2	Set a weekly working schedule (e.g. Mon–Sat 9:00–18:00, Sun off).	Appointment booking outside these hours for this employee is rejected (see 5.1).
☐	3	Deactivate an employee.	No longer bookable; past appointments/commission history stay intact.

3. Phase 3 — Customer CRM

3.1 Customer CRUD

✓	#	Action	Expected result
☐	1	Create a customer (name, phone, email, tags, preferences).	Saved; appears in customer search by name/phone/email.
☐	2	Add an internal note to the customer.	Note saved with timestamp, visible on profile.
☐	3	Record marketing consent (opt in), then opt out.	Consent ledger shows both events with timestamps — not just a single flag.

3.2 DPDP Erasure Workflow

✓	#	Action	Expected result
☐	1	Give the customer at least one finalized invoice (Phase 6) first, then request erasure.	Customer's name/contact are anonymized ("Erased Customer" etc.).
☐	2	Open the earlier invoice again.	Invoice still shows the original customer name/phone (snapshotted at sale time) — erasure never corrupts financial history.

4. Phase 4 — Service Catalogue & Vertical Consultations

4.1 Categories, Services, Variants

✓	#	Action	Expected result
☐	1	Create a service category under a module (e.g. Salon → "Hair").	Rejected if you pick a module the tenant hasn't enabled.
☐	2	Create a service in it (duration, price, tax %).	Saved; service inherits the category's module.
☐	3	Add two variants with different prices (e.g. Short hair / Long hair).	Both selectable at booking time with their own price/duration.
☐	4	Make the service available at a branch with a price override.	That branch charges the override price, others use the base price.
☐	5	Assign staff capability (which employees can perform it).	Only capable employees appear when booking this service.
☐	6	Try creating a second service with the exact same name in the same tenant.	Rejected — name must be unique per tenant (but same name is fine in a different tenant).

4.2 Vertical Consultations

module-gated

✓	#	Action	Expected result
☐	1	From a customer's profile (with Salon module enabled), open Hair Profile, fill it in.	Saved and shown on later visits.
☐	2	Record a Hair Consultation (concerns, recommendation, colour formula).	Appears in consultation history, newest first.
☐	3	Repeat for Skin Profile/Consultation (Beauty Parlour module) and Spa Profile/Consultation (Spa module) if those modules are enabled.	Each vertical's consultation is independent and only visible/creatable when that module is enabled for the tenant.
☐	4	Disable the Spa module for the tenant (Platform), then revisit a customer's Spa consultation page.	Access blocked even by direct URL, not just hidden from nav.

5. Phase 5 — Appointment Engine

5.1 Booking & availability

✓	#	Action	Expected result
☐	1	Book an appointment: pick branch, customer, service, employee, a valid future time slot.	Created with status <i>Confirmed</i> .
☐	2	Try booking outside the branch's business hours (2.1) or on a holiday (2.2).	Rejected with a clear conflict message — not a generic error.
☐	3	Try booking outside the employee's own weekly schedule (2.4).	Rejected.
☐	4	Book the same employee for two overlapping time slots (open two tabs / submit twice quickly).	The second booking is rejected — no double-booking allowed, even under concurrent submission.
☐	5	If a resource (2.3) was selected, try double-booking that same resource at an overlapping time with a different employee.	Rejected — resource conflicts are checked independently of employee conflicts.

5.2 Lifecycle

✓	#	Action	Expected result
☐	1	Check in the customer.	Status → Checked-in.
☐	2	Start the service.	Status → In-service.
☐	3	Complete it.	Status → Completed. Now eligible for invoicing (Phase 6), product-consumption recording (Phase 7), and package redemption (Phase 8).
☐	4	On a different appointment, mark No-show instead.	Status → No-show; cannot be invoiced.
☐	5	On a pending/confirmed appointment, reschedule to a new valid time.	Time updates, same conflict checks as a fresh booking apply.
☐	6	Cancel an appointment with a reason.	Status → Cancelled, reason stored.

5.3 Waitlist

✓	#	Action	Expected result
☐	1	Add a customer to the waitlist for a service/branch with no specific time.	Appears in the waitlist queue.
☐	2	Convert a waitlist entry into a real booking.	Appointment created, waitlist entry marked booked.

6. Phase 6 — POS, Invoicing, Payments, Refunds

6.1 Building a sale

✓	#	Action	Expected result
☐	1	Start a new sale for a customer at a branch.	Draft invoice created, no invoice number yet.
☐	2	Add a line from a completed appointment (5.2).	Price auto-fills from the appointment's agreed price, not the current catalogue price.
☐	3	Try adding the <i>same appointment</i> to a second invoice.	Rejected — an appointment can only be invoiced once.
☐	4	Add a standalone walk-in line (service picked directly, no appointment) with a manual discount.	Discount capped at the line's own value; CGST+SGST computed from the service's tax %.
☐	5	Remove a line before finalizing.	Totals recalculate.

6.2 Checkout & GST numbering

✓	#	Action	Expected result
☐	1	Try finalizing a draft with zero lines.	Rejected (422) — cannot finalize an empty sale.
☐	2	Finalize ("checkout") the sale.	Gets a sequential invoice number like <code>{branch code}/{financial year}/{seq}</code> .
☐	3	Finalize a second sale at the same branch.	Sequence number increments by exactly 1, no gaps or repeats.

6.3 Payments

✓	#	Action	Expected result
☐	1	Pay half the invoice in cash.	Status → Partially Paid.
☐	2	Pay the remaining balance via a second method (e.g. UPI).	Status → Paid. Split payment across two methods recorded correctly.
☐	3	Try paying more than the remaining balance.	Rejected (409) — cannot overpay.
☐	4	Double-click / resubmit the same payment form quickly.	Only one payment recorded — idempotency key prevents duplicates.
☐	5	Add a tip amount alongside a payment.	Tip recorded separately, does not affect the invoice's taxable balance.

6.4 Void & Refund

✓	#	Action	Expected result
			Owner only

☐	1	Finalize a new invoice, then void it before any payment.	Status → Void. Its invoice number is still "used up" (GST requires unbroken numbering).
☐	2	Pay an invoice fully, then try to void it.	Rejected (409) — a paid invoice can only be refunded, never voided.
☐	3	Issue a partial refund on a paid invoice.	Status stays Paid until the full paid amount has been refunded.
☐	4	Refund the remaining balance.	Status → Refunded.
☐	5	Try refunding more than what's actually been paid.	Rejected (409).
☐	6	As a Manager, try Void/Refund.	403 Forbidden — Owner-only by default.

7. Phase 7 — Products, Inventory & Purchasing

7.1 Catalogue

✓	#	Action	Expected result
☐	1	Create a product category, then a product in it (unit, cost price, reorder level).	Saved. Products are shared Core catalogue — not tied to any one module.
☐	2	Create a supplier.	Saved.

7.2 Purchase Order lifecycle

✓	#	Action	Expected result
☐	1	Create a PO for a branch/supplier with one product line, quantity 100.	Status Draft, no stock movement yet.
☐	2	Place the order.	Status → Ordered.
☐	3	Receive 60 units against it.	Status → Partially Received. Branch stock for that product increases by 60.
☐	4	Receive the remaining 40.	Status → Received. Stock now at 100.
☐	5	Try receiving more against the same (now-complete) line.	Rejected (409).
☐	6	Create a second PO and cancel it while still in Draft/Ordered.	Cancelled successfully.
☐	7	Try cancelling a fully-received PO.	Rejected — cancel is only valid before goods are received.

7.3 Returns, adjustments, transfers

✓	#	Action	Expected result
☐	1	Record a purchase return of 10 units to the supplier.	Stock decreases by 10.
☐	2	Try returning more than what's in stock.	Rejected — cannot go negative.
☐	3	Record a "damage" stock adjustment for 5 units.	Stock always decreases for damage/expiry, regardless of sign entered.
☐	4	Record a manual "adjustment" with direction = increase, then one with direction = decrease.	Stock moves in the chosen direction each time.
☐	5	Transfer 20 units from Branch A to Branch B.	Branch A stock -20, Branch B stock +20, in one atomic action.
☐	6	Try transferring more than Branch A has.	Rejected — source branch cannot go negative.
☐	7	Open the Movement Ledger for a branch.	Every stock change above appears as its own dated, typed entry (purchase/return/damage/adjustment/transfer).

7.4 Service consumables

✓	#	Action	Expected result
☐	1	On a service, add a consumable (e.g. "10ml Shampoo per haircut").	Saved as the service's "recipe".
☐	2	Complete an appointment for that service, then click "Record product usage".	Branch stock for Shampoo decreases by 10ml.
☐	3	Click it again on the same appointment.	Rejected — usage already recorded for this appointment.

8. Phase 8 — Packages, Membership, Loyalty, Wallet, Gift Cards

Note: Package/Membership/Gift Card *purchases* are recorded as simple payment-captured sales, not full GST invoices — this is a documented, deliberate scope decision (see project decisions log D-006), not a bug.

8.1 Packages

✓	#	Action	Expected result
☐	1	Create a package template, e.g. "2 Haircuts" for ₹900, 90-day validity, containing 2× Haircut.	Saved.
☐	2	From a customer's page, sell them this package (cash).	Customer now owns an active package with 2 haircuts remaining, expiring in 90 days.
☐	3	Book & complete a Haircut appointment for that customer, then redeem 1 unit from the package against it.	Remaining count drops to 1; package still Active.
☐	4	Redeem the last unit on a second completed appointment.	Package status flips to Exhausted.
☐	5	Try redeeming again.	Rejected (409) — no remaining quantity / not active.
☐	6	Sell another package, then cancel it before it's used.	Status → Cancelled.

8.2 Memberships

✓	#	Action	Expected result
☐	1	Create a membership plan, e.g. "Gold" at 20% off, ₹5000, 365-day validity, no service/branch/module restriction.	Saved.
☐	2	Sell it to a customer.	Customer now has an active Gold membership.
☐	3	Start a new sale for that customer, add a service line, and select their membership from the dropdown instead of a manual discount.	Line discount auto-computes as 20% of the service price. Manual discount field is disabled/rejected in combination.
☐	4	Check the membership's usage count afterwards.	Incremented by 1.
☐	5	Restrict a plan to a specific branch/service via its Applicability settings, then try applying it on an ineligible service/branch.	Rejected — membership doesn't apply there.

8.3 Loyalty

✓	#	Action	Expected result
☐	1	In Settings, set Loyalty to e.g. 1 point per ₹100 spent, redemption value ₹0.50/point.	Saved; loyalty now active tenant-wide.
☐	2	Finalize and pay an invoice of, say, ₹590 in cash.	Customer automatically earns 5 points (floor(590/100)) — no manual step needed.

☐	3	On a new invoice, redeem 3 points as a payment method.	₹1.50 applied toward the balance; points balance drops by 3.
☐	4	Try redeeming more points than the customer has.	Rejected (409).
☐	5	View the customer's Loyalty page.	Full ledger of every earn/redeem event with dates.

8.4 Wallet

✓	#	Action	Expected result
☐	1	From a customer's Wallet page, add a manual credit of ₹200.	Balance shows ₹200.
☐	2	On an invoice, redeem ₹50 from the wallet as a payment method.	Balance drops to ₹150; invoice balance reduces by ₹50.
☐	3	Try redeeming more than the remaining wallet balance.	Rejected (409).

8.5 Gift Cards

✓	#	Action	Expected result
☐	1	Issue a gift card for ₹300 (optionally tied to a customer).	A unique code is generated (never entered manually).
☐	2	On an invoice, redeem ₹100 using the gift card's code.	Balance drops to ₹200.
☐	3	Try redeeming more than the remaining balance.	Rejected (409).
☐	4	Cancel the gift card (Owner only).	Status → Cancelled, remaining balance written off to zero. As Manager, this action should be 403.

8.6 Refund reversal to Wallet / Loyalty

✓	#	Action	Expected result
☐	1	On a paid invoice, issue a refund choosing method = Wallet.	Customer's wallet balance increases by the refund amount instead of handing back cash.
☐	2	Issue another refund choosing method = Loyalty.	Customer's points balance increases by (amount ÷ redemption value).

9. Role & Permission Matrix

Use this to spot-check that each role sees exactly what it should — both in the sidebar (hidden) and by direct URL (blocked with 403).

Area	Owner	Manager	Staff
Customers / Appointments	Full	Full	View + create/update (no delete)
Invoices — create, record payment	Full	Full	No access
Invoices — void / refund	Full	403 Forbidden	403 Forbidden
Products / Inventory / Suppliers / Purchase Orders	Full	Full operational access	View only
Products — delete (deactivate)	Full	403 Forbidden	403 Forbidden
Supplier payments	Full	403 Forbidden	403 Forbidden
Packages / Memberships — sell, redeem	Full	Full	View only
Packages / Memberships — delete template	Full	403 Forbidden	403 Forbidden
Wallet credit / Gift card issue	Full	Full	View only
Gift card cancel	Full	403 Forbidden	403 Forbidden
Branches / Employees / Settings	Full	View mostly	No access

10. Cross-Cutting Checks

10.1 Tenant isolation (do this for at least 2–3 modules)

✓	#	Action	Expected result
☐	1	Create Tenant A and Tenant B (two separate <code>/register</code> signups). Create a customer/invoice/appointment under Tenant A.	—
☐	2	Log in as Tenant B's owner. Guess Tenant A's record URL directly (e.g. <code>/customers/7/edit</code> , <code>/invoices/3</code>).	404 Not Found every time — never Tenant A's data, never a generic error page leaking details.

10.2 Financial integrity

✓	#	Action	Expected result
☐	1	Try to edit a finalized invoice's line items directly (there should be no such option in the UI once finalized).	No edit control exists — only void/refund workflows change a finalized invoice's outcome.
☐	2	Compare a branch's current stock quantity against summing its Movement Ledger by hand for one product.	They match — stock is always reconstructable from the ledger.

10.3 UI / responsiveness

✓	#	Action	Expected result
☐	1	Resize the browser to a phone width (or open on an actual phone).	Sidebar collapses behind a hamburger icon; all pages remain usable, no horizontal scroll.
☐	2	Check the Dashboard as Owner vs. as Staff.	Cards you don't have permission for (e.g. Sales) simply don't render for Staff — no error.